METAVUL: AGENTIC HARNESS DISCOVERY FOR VULNERABILITY DETECTION

Ira Ceka[1], **Arnav Garg**[1], **Ryan Lutz**[2], **Ting Zhang**[3], **Ratnadira Widyasari**[4], **Gail Kaiser**[5], **Baishakhi Ray**[1]

[1]Columbia University [2]The Citadel [3]Monash University
[4]University of Victoria [5]Columbia University, Department of Computer Science
iceka@cs.columbia.edu ag5516@columbia.edu rlutz@student.citadel.edu
Ting.Zhang@monash.edu ratnadiraw@uvic.ca
kaiser@cs.columbia.edu rayb@cs.columbia.edu

ABSTRACT

Large Language Model (LLM) agents are increasingly used to automate vulnerability detection, but reliable detection is challenging because agents must distinguish subtle, security-relevant fixes from benign code variation. While underlying LLM capability has improved significantly, the accuracy of the agent still depends heavily on the *harness*: the code that defines the scaffolding around the LLM. Harnesses are typically fixed, and largely hand-written, limiting adaptation to evaluator feedback. Additionally, when harnesses are static, improvements on one dataset or model may not necessarily transfer to others. To address these challenges, we introduce MetaVul, a self-improving framework for harness discovery in vulnerability detection. Starting from a seed structured reasoning scaffold, MetaVul uses a workflow-based optimization loop to iteratively refine the harness using feedback from paired vulnerable and patched functions, all while preserving the structured reasoning and verdict format. At each iteration, it searches over the full history of prior candidate harnesses, with access to their trajectories, logs, and metrics, and it returns a set of Pareto-optimal harnesses rather than a single best candidate. We apply MetaVul to discover agent harnesses for architectures including Codex CLI and Claude Code. A validation set (202 pairs) from TitanVul and CWEval is used for optimization, while the unseen PrimeVul dataset (430 pairs) is kept as the held-out set. We later test whether harnesses learned from one model transfer to others not seen during optimization. Results on both frontier and small LLMs show that the optimized harnesses improve pairwise vulnerability detection up to 23.3 percentage points on the validation set (Opus 4.6) and up to 25.8 points on the held-out set with a Pareto-frontier harness (DeepSeek V4 Flash). Harnesses learned with one model also transfer to others. We show that harnesses optimized with Opus 4.6 transfer to Opus 5, improving held-out pairwise accuracy by 14.7 points without the need for further optimization. Code is available at `https://github.com/ARiSE-Lab/MetaVul` (ARiSE Lab, 2026).

1 INTRODUCTION

Artificial intelligence (AI) agents utilize a large language model (LLM) as their core reasoning component for taking actions toward a given goal. "Vibe coding," or using AI tools, particularly by way of coding agents, is becoming increasingly prevalent across the software industry. Consequently, the growing volume and pace of AI-assisted code generation increase the importance of effective vulnerability detection. LLM agents rely not only on the capabilities of their underlying model, but also on the harnesses that structure how those agents reason, interact with tools, and arrive at a decision. Designing these harnesses, known as *harness engineering*, has become a practice of its own (Lopopolo, 2026; Rajasekaran, 2026). For vulnerability detection, these harnesses are typically hand-designed and are fixed during deployment, even though their structure and context can substantially alter an agent's results. In practice, vulnerability detection is substantially affected by how the agent inspects the code, and how the harness will shape that workflow (Yildiz et al., 2025).

This understanding directly motivates a harness that can learn its design systematically through evaluation feedback.

Recent work studies the evaluation, evolution, and adaptation of agent harnesses using graded scores, verifiers, execution traces, or prior experience (Lee et al., 2026; Ursekar et al., 2026; Huang et al., 2026a; Zhang et al., 2026; Huang et al., 2026b; Esakkiraja et al., 2026; Nie et al., 2026), while OneDayAgent develops a fixed long-horizon harness (Zheng et al., 2026); however, none investigates harness optimization for vulnerability detection or incorporates security-specific objectives such as CWE identification, vulnerability localization, and vulnerable-patched discrimination. Harness engineering for vulnerability detection, however, poses challenges.

Firstly, **false alarms are costly.** In practical security settings, analysts receive far more alerts than they can effectively triage. Several industry studies identify alert fatigue and analyst burnout as major problems (Tariq et al., 2025). This problem is well documented among traditional vulnerability-detection tools (Charoenwet et al., 2024; Sajadi et al., 2026) and persists with recent LLM-based detectors (Li et al., 2026a). As recently as April 2026, an independent security company reported poor discriminative performance on PrimeVul (Ding et al., 2024)[1]; even Claude Opus 4.6, a flagship model shown by Anthropic to discover previously unknown vulnerabilities Carlini et al. (2026), struggled to reliably distinguish vulnerable functions from their patched counterparts (Walker, 2026). These findings further our motivation and demonstrate the outstanding and relevant nature of vulnerability discrimination, which remains an open and significant problem in practice. Moreover, these findings reinforce the notion that strengthening the underlying models does not eliminate the need for a robust design surrounding the agentic workflow. Rather, harnesses and their associated design choices matter as they directly affect downstream behavior.

Secondly, **harness optimization is non-trivial.** A vulnerability detection harness controls how an agent inspects code, how it structures its analysis, and how it reasons about security. Vulnerability detection as a task, however, poses a *multi-objective* challenge. A strong harness not only has to accurately classify vulnerabilities, but also localize them, identify their CWE(s), ground the conclusions in code evidence, as well as distinguish the vulnerable functions from their patched counterparts. Ding et al. (2024) show why traditional classification metrics, such as accuracy and F1, are insufficient on their own: models that perform well on conventional benchmarks approach random guessing on PrimeVul's paired evaluation. Optimizing for a single metric may therefore reward superficial patterns rather than robust vulnerability reasoning. HarnessDev (Wu et al., 2026) shows that a harness can overfit to the model that wrote it, memorize development examples, or improve one capability while silently regressing another. It can also overfit its search tasks: Wang et al. (2026) found that, when harness evolution and evaluation share the same tasks, the resulting improvements largely disappear on held-out tasks, which suggests that optimization can learn task-specific shortcuts rather than generalizable harness improvements.

Finally, **defense must keep pace with AI-driven offense.** With the rise of AI-driven exploit generation, attackers are moving at immense speed. In October 2026 alone, two critical PHP vulnerabilities were disclosed after AI-assisted research revealed that a 14-year-old memory-safety flaw in the PHP runtime engine allowed unauthenticated attackers to execute malicious code remotely on servers hosting PHP-based web applications, triggering a global security crisis (Wiz Research, 2026). These attacks involve auditing millions of lines of code, mapping out reliable exploit strategies, and adapting to multiple configurations. While offensive security is bottlenecked by compute, defense has another challenge to tackle: to survive this offensive scale, defenders must automate triage and blocking, but doing so can be catastrophic due to the false positives mentioned previously. It is thus paramount for defense to have scalable methods for identifying suspicious code and evaluating, triaging, and applying potential fixes quickly.

We address these challenges with **MetaVul**, which improves vulnerability detection through iterative harness construction, driven by validation feedback. The "**meta**" in MetaVul comes from the way MetaVul finds better ways to find vulnerabilities, with the optimizer and detector operating on two different levels: the optimizer designs the harness, and the detector uses it. A sandboxed detection agent evaluates each candidate harness during the discovery phase, and MetaVul returns a Pareto frontier of harnesses that trade off discrimination and localization ability. The discovered harnesses

[1] Following PrimeVul, we adopt *pairwise accuracy* as a core metric alongside accuracy and F1. High pairwise accuracy indicates that the detector can distinguish vulnerable code from its fixed version, reducing false alerts and, consequently, alert fatigue. We explain this further in Sec. 4.2.

are then evaluated against held-out datasets and unseen models, hence testing generalization. We ask whether MetaVul improves vulnerability-detection harnesses and whether the gains transfer to unseen data and unseen models (**RQ1**). We then evaluate how MetaVul compares with other LLM-based or multi-agent vulnerability detection methods (**RQ2**). Finally, we ask whether MetaVul's gains rely on "shortcut" exploitation, like data leakage signals, embedded answers, or trivial verdict bias (**RQ3**). We find that MetaVul improves pairwise accuracy by 11.4 to 23.3 points ($1.6\times$-$4.3\times$) on validation and by up to 25.8 points ($2.9\times$) on 430 unseen PrimeVul pairs. Harnesses also transfer to models that took no part in optimization, with gains up to 14.7 points ($1.7\times$). MetaVul outperforms prior LLM-based and multi-agent detectors and does so without evidence of "shortcut" exploitation.

2 Related Work

Previous work has explored LLM-based vulnerability detection, harness engineering and optimization, and structured reasoning strategies for code. **LLM-based vulnerability detection.** Ding et al. (2024) show that LLMs struggle to distinguish vulnerable functions from their patched versions under paired evaluation. They propose pairwise accuracy as a stricter alternative to conventional metrics, and also introduce PrimeVul, which addresses the noisy labels and train-test duplication of earlier datasets through stricter labeling, de-duplication, and a chronological split. Chain-of-thought (CoT) prompting (Wei et al., 2022) improves pairwise performance over two-shot prompting but stays below random guessing. GPTLens (Hu et al., 2023) pairs an auditor that proposes candidate vulnerabilities with a critic that filters false positives, VulTrial (Widyasari et al., 2026) makes use of courtroom-style agents that debate a verdict, and ConfColl (Tsai et al., 2025) escalates from a single agent to retrieval-augmented and multi-agent reasoning when needed. All four methods use GPT-4o as the backbone (OpenAI, 2024; Tang et al., 2026), and we compare against them in Sec. 6. Unlike GPTLens, VulTrial and ConfColl, MetaVul keeps a single detector agent and instead optimizes its harness. At project scale, Li et al. (2026a) report low recall, high false-discovery rates and high computational cost. **Harness engineering and optimization.** Recent work treats the agent harness (prompts, tools, memory, context management and control flow) as an optimization target independent of model weights (Lee et al., 2026; Ursekar et al., 2026; Lopopolo, 2026; Rajasekaran, 2026), in contrast to weight-level methods such as ORPO (Hong et al., 2024). GEPA (Agrawal et al., 2026) optimizes prompts from execution trajectories and model feedback, and SecureForge (Liu et al., 2026) optimizes system prompts for secure code generation using automated vulnerability auditing. Meta-Harness, MemoHarness, DarwinX, StarHarness, and TTHE explore complementary forms of harness adaptation using execution experience, search, held-out selection, and test-time adaptation (Lee et al., 2026; Huang et al., 2026b; Zhang et al., 2026; Esakkiraja et al., 2026; Nie et al., 2026), and OneDayAgent supports long-horizon execution with task decomposition, persistent memory and output verification (Zheng et al., 2026). HarnessOpt-Bench, Evo-Bench and HarnessDev evaluate whether models can build or improve harnesses that generalize beyond their optimization tasks (Ursekar et al., 2026; Huang et al., 2026a; Wu et al., 2026), and Wang et al. (2026) underscore the need for held-out evaluation, showing that any apparent gains might vanish on held-out tasks, or simply reflect that extra compute was spent searching over candidates. **Reasoning strategies for code.** Structured chain-of-thought (SCoT) organizes reasoning around program structures (Li et al., 2023), Modularization-of-Thought (MoT) decomposes it into hierarchical modules (Pan & Zhang, 2025), and Tree of Thoughts (ToT) explores multiple reasoning paths (Yao et al., 2023). Despite this, to our knowledge, none of these methods have explored harness optimization for vulnerability detection, nor tested whether such harnesses transfer to unseen datasets and models. MetaVul automatically optimizes harnesses for this task, selects them on security-specific objectives (telling vulnerable functions apart from their fixes, i.e., pairwise accuracy, and joint CWE and line localization), and evaluates them on an unseen dataset and on models that took no part in optimization.

3 Methodology

MetaVul separates harness discovery from evaluation on unseen data and models. Beginning with a seed harness (h_0), the optimizer ($\mathcal{A}_{\text{opt}}$) proposes a new harness, using *validation feedback*. The isolated detector ($\mathcal{A}_{\text{det}}$) evaluates each harness candidate. After discovery, MetaVul retains the Pareto-optimal harnesses and evaluates them on held-out PrimeVul using the search and transfer models. Fig. 1 presents the full pipeline, and Algorithm 1 details the discovery loop.

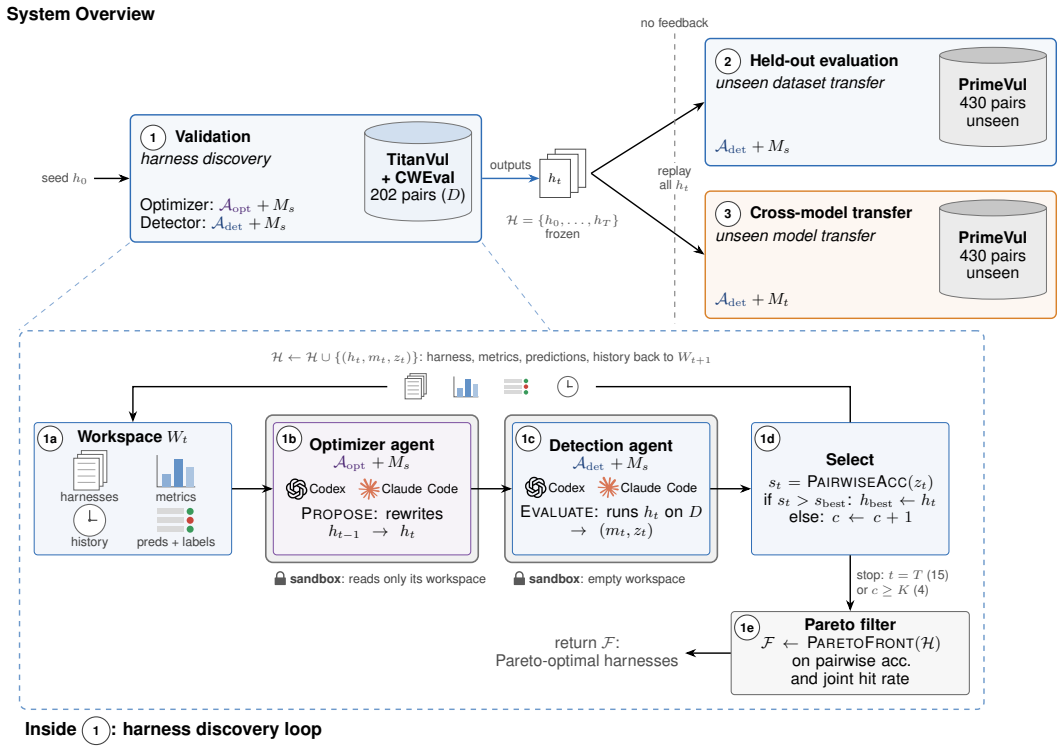


Figure 1: **MetaVul pipeline.** (1) Starting from seed h_0, harnesses are discovered on the validation set. In each iteration, (1a) the workspace stores previous harnesses and results, (1b) $\mathcal{A}_{\text{opt}}$ proposes a new harness h_t, (1c) $\mathcal{A}_{\text{det}}$ evaluates it, and (1d) the best result is selected. The Pareto filter (1e) retains harnesses based on pairwise accuracy and joint hit rate. Each harness is then frozen and tested on unseen PrimeVul using (2) the search model M_s and (3) a different transfer model M_t. PrimeVul provides no optimization feedback.

Harness discovery and selection. MetaVul iteratively generates and evaluates candidate harnesses before selecting those that best balance its evaluation objectives. ***Discovery loop.*** MetaVul first evaluates the seed harness h_0. At each iteration t, the optimizer uses previous harnesses and results to propose h_t, which the detector evaluates on D. The results are added to the search history. Discovery ends after T iterations or K iterations without improving pairwise accuracy. ***Agent roles.*** The optimizer $\mathcal{A}_{\text{opt}}$ revises the harness using validation feedback. The detector $\mathcal{A}_{\text{det}}$ runs the proposed harness and returns predictions for scoring. The agents may use the same model, but they run separately and never share state. ***Pareto selection.*** After the loop, MetaVul returns the Pareto frontier of all evaluated harnesses over two validation objectives, pairwise accuracy and joint hit rate (Sec. 4.2), both maximized (PARETOFRONT in Algorithm 1). Supp. S6 gives a worked example. We say harness A *dominates* harness B if A scores no lower than B on both pairwise accuracy and joint hit rate, and higher than B on at least one of those metrics. The frontier $\mathcal{F}$ contains those harnesses which are not dominated by any other harness. If two or more harnesses have identical scores on both metrics, we keep only the earliest one.

Sandboxing. It is essential that the held-out set is never exposed to the agent during discovery (the "optimization" phase); otherwise, contamination would make any held-out gains indistinguishable from memorization. Neither discovery nor evaluation should be influenced by system files, default instruction files (such as `CLAUDE.md`) or user settings. To address this, we sandbox every agent. The optimizer and detector agents each run isolated CLI processes that can neither request nor receive any additional permissions. Each agent is limited in the data that it can access. As shown in Fig. 1, the agent and its model serve two roles: an optimizer and a detector (which we also call the evaluator). ***Optimizer.*** The optimizer can only access its workspace, which holds the validation history: earlier harnesses, their metrics, and their predictions with labels. After each iteration, its results are added to the workspace of the next iteration. Supp. S10 describes the context budget. For Claude Code, the working directory is set to the workspace, the read tool is restricted to it, write and edit tools

Algorithm 1 METAVUL harness discovery loop

Require: Seed harness h_0, validation pairs D, maximum iterations T, patience K, optimizer agent $\mathcal{A}_{\text{opt}}$, detection agent $\mathcal{A}_{\text{det}}$

$(m_0, z_0) \leftarrow \text{EVALUATE}(\mathcal{A}_{\text{det}}, h_0, D)$ ▷ m: metrics; z: predictions
$\mathcal{H} \leftarrow \{(h_0, m_0, z_0)\}$; $h_{\text{best}} \leftarrow h_0$
$s_{\text{best}} \leftarrow \text{PAIRWISEACC}(z_0)$; $c \leftarrow 0$
for $t \leftarrow 1$ to T **do**
  $W_t \leftarrow \text{BUILDWORKSPACE}(\mathcal{H}, h_{t-1})$
  $h_t \leftarrow \text{PROPOSE}(\mathcal{A}_{\text{opt}}, W_t)$ ▷ keeps SCoT and verdict schema
  if h_t is invalid or the optimizer fails **then**
    $c \leftarrow c + 1$; **continue**
  end if
  $(m_t, z_t) \leftarrow \text{EVALUATE}(\mathcal{A}_{\text{det}}, h_t, D)$ ▷ fresh call per function
  $s_t \leftarrow \text{PAIRWISEACC}(z_t)$
  $\mathcal{H} \leftarrow \mathcal{H} \cup \{(h_t, m_t, z_t)\}$
  if $s_t > s_{\text{best}}$ **then**
    $h_{\text{best}} \leftarrow h_t$; $s_{\text{best}} \leftarrow s_t$; $c \leftarrow 0$
  else
    $c \leftarrow c + 1$
  end if
  if $c \geq K$ **then break**
  end if
end for
$\mathcal{F} \leftarrow \text{PARETOFRONT}(\mathcal{H};\ \text{PAIRWISEACC}, \text{JOINT})$ ▷ non-dominated
return $\mathcal{F}$

Table 1: Datasets used by MetaVul. A circled number corresponds to its respective pipeline stage in Fig. 1.

Validation set D	
Dataset	TitanVul + CWEval
Pairs	202
Languages	161 C, 41 C++
CWEs	72
Used for	① Validation

Held-out set	
Dataset	PrimeVul
Pairs	430
Languages	C/C++
CWEs	61
Used for	② Held-out evaluation
	③ Cross-model transfer

are denied, and shell access is limited to read-only commands. No `CLAUDE.md` files, settings, or memory related to the user or project are loaded into the context. For Codex, an OS-level sandbox grants read-only access to the workspace, with no write or network access. ***Detector.*** The detector is launched freshly for every function. To ensure a non-contaminated run, it has no workspace access (i.e., from the optimizer), no labels, no trajectory, and no memory between calls. The detector receives only the rendered harness and one function. For this phase, Claude Code runs from a `/tmp` directory with no tools and an empty system prompt. Similarly, Codex runs the harness in an empty temporary directory tree with read-only access and no network connection. The predictions from the held-out PrimeVul evaluation and the cross-model transfer are never added to any workspace (only the harness changes between iterations). Detailed agent restrictions are given in Supp. S8.

4 EXPERIMENTAL DESIGN

We outline our experimental setup: the datasets, seed harness, agents, models, and configurations (Sec. 4.1), and the evaluation metrics (Sec. 4.2).

4.1 DATASETS, MODEL, AND AGENT SELECTION

Datasets. We use separate datasets for optimization and testing, so that we can measure how harness improvements carry over to an unseen dataset and to models that took no part in optimization. Every sample is a vulnerable function paired with its patched version. Table 1 summarizes both splits.

- **Validation set** D**.** We handcraft a dataset D from TitanVul and CWEval. TitanVul (Li et al., 2026b) combines seven public vulnerability datasets and removes duplicate samples. Multiple LLM agents check fixes, keeping those that address genuine vulnerabilities understandable from the function alone. These checks reduce incorrect labels and unrelated code changes. We select 150 C/C++ pairs, sampled by CWE in proportion to the CWE's frequency (Supp. S1). We enforce some strict criteria, such as ensuring each CWE receives a proportional share to its TitanVul occurrence, there is a minimum of one of every CWE, even if rare, and pairs with unlabeled CWEs are also considered. CWEval (Peng et al., 2025) evaluates both the functional correctness and the security of LLM-generated code; each of its security-critical tasks has functional and security tests and a secure and an insecure reference implementation. We use all 52 of its C and C++ tasks, pairing each insecure implementation with its secure one. Together these form D: 202

pairs (404 functions). Because 79 of the 150 TitanVul pairs have no CWE label, CWE accuracy on D is computed over the remaining pairs which do.

- **Held-out set.** PrimeVul (Ding et al., 2024) addresses several issues of earlier datasets, including noisy labels and duplicate functions. PrimeVul contains vulnerable functions and their fixes from real open-source C/C++ projects, and we use its paired test set as our held-out "unseen" set, as this is a hard benchmark on which LLMs reach low pairwise accuracy. Because TitanVul aggregates public vulnerability datasets, we checked it for overlap with D: 5 PrimeVul pairs contain the same functions and fixes as TitanVul pairs in D. We remove them, leaving 430 pairs (860 functions) from 141 projects. Each vulnerable function in PrimeVul has a CWE label. The optimizer never sees PrimeVul data.

Seed harness. The harness is a standalone Python file that prepares a function for the detection agent and parses the agent's verdict. Every run begins with the same 151-line SCoT seed harness (Li et al., 2023), provided in Supp. S3.[2]

Agent selection. We use Codex CLI and Claude Code. In each discovery session, the optimizer $\mathcal{A}_{\text{opt}}$ and detector $\mathcal{A}_{\text{det}}$ run the same CLI agent and model (only their access differs). The optimizer can read earlier harnesses, metrics, and labeled predictions in its workspace. The detector sees only the harness and one function (Sec. 3). For unseen-dataset and cross-model transfer, only the detector ($\mathcal{A}_{\text{det}}$) runs, using the frozen harnesses to evaluate PrimeVul.

Model selection. We optimized harnesses with Inkling-Small (Thinking Machines Lab, 2026), DeepSeek V4 Flash (OpenRouter, 2026a), Opus 4.6 and GLM-5.3 (OpenRouter, 2026b). We tested cross-model transfer on Opus 5, Nemotron 3.5 Lightning (Thinking Machines Lab, 2026; OpenRouter, 2026f), GPT-4o (OpenRouter, 2026d), GLM-4.7 Flash (OpenRouter, 2026c) and GPT-OSS-120B (OpenRouter, 2026e). These models cover smaller and frontier models, including open-weight and closed models. Opus 4.6 ran through our Claude subscription, whose usage limits every optimizer and detector call draws on, so we chose the remaining models from per-token providers (OpenRouter and Tinker; Supp. S12).

Configurations. Opus 4.6 and Opus 5 ran through our Claude Code subscription, while all other models were applied through a local LiteLLM proxy to our external model providers (OpenRouter or Tinker). All runs allow up to 15 refinement iterations. Inkling-Small and DeepSeek V4 Flash ran all 15; GLM-5.3 used a patience of 4 and stopped after iteration 8; Opus 4.6 ran four, limited by subscription usage. Additional serving, timeout, and decoding configurations are outlined in Supp. S9.

4.2 METRICS

We evaluate each harness on its detection, discrimination, and localization ability. ***Traditional detection metrics.*** We treat each function as a binary decision with vulnerable as the positive class, and report accuracy, precision, recall and F1 over all functions. ***Pairwise accuracy.*** Let pair i contain a vulnerable function x_i^v and its patched version x_i^p. For a binary detector f, pairwise accuracy over N pairs is $\text{PairAcc}(f) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[f(x_i^v) = 1 \wedge f(x_i^p) = 0]$. A pair counts only when both decisions are correct, which makes the metric stricter than ordinary accuracy.[3] Pairwise accuracy is the target metric used in Algorithm 1. ***CWE and localization metrics.*** These score only true positives, the vulnerable functions i that the harness flags, and only those with the relevant ground truth: a set of CWEs C_i (usually one, sometimes several) and the gold lines G_i. None of the three datasets marks which lines are vulnerable, so we derive G_i from each pair: it contains the lines of the vulnerable function that are modified or deleted by the fix, or the lines around the insertion point if the fix only adds lines.[4] For CWEval, the fix is the secure reference implementation. For each such function, the harness returns an ordered list of CWE guesses, whose first guess is $\hat{c}_i^{(1)}$, and a set of evidence lines $\hat{G}_i$. *CWE accuracy* is the percentage whose CWE list contains a CWE in C_i. *Loc F1* is the harmonic

[2] In preliminary experiments, SCoT achieved higher accuracy and precision than a plain instruction, MoT, and ToT. Its structure also supports reasoning about sequences, branches, loops, and patch effects. [3] A detector that labels both the vulnerable function and its patch as vulnerable gets 50% accuracy and full recall, but $\text{pAcc} = 0$. Hence, pAcc evaluates the extent to which the detector is able to identify the fix. Supp. S4 gives a worked example with three detectors. [4] The datasets pair vulnerable code with a benign associated counterpart. We make use of the lines changed by the fix as the closest approximation of the vulnerability position.

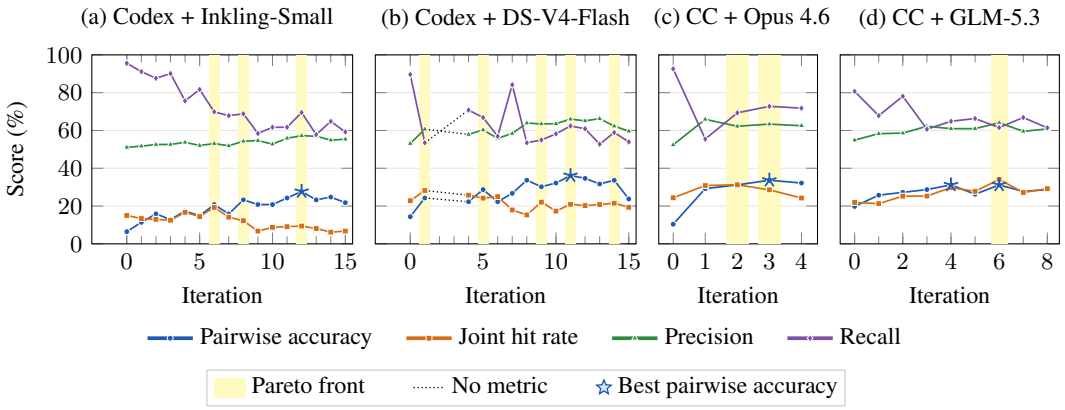


Figure 2: **Validation metrics across optimization.** Each panel shows pairwise accuracy, joint hit rate, precision, and recall on TitanVul+CWEval for one agent-model configuration (CC = Claude Code; DS = DeepSeek).[7] Pairwise accuracy generally rises with precision while recall declines, indicating fewer false alarms but more missed vulnerabilities. Joint hit rate varies by model. marks the Pareto front and ☆ the best pairwise accuracy.

mean of line-level precision $|\hat{G}_i \cap G_i|/|\hat{G}_i|$ and recall $|\hat{G}_i \cap G_i|/|G_i|$, each averaged over functions, and *loc hit rate* is the percentage with at least one correct line ($\hat{G}_i \cap G_i \neq \emptyset$). ***Joint hit rate.*** Joint (JOINT in Algorithm 1) asks whether a correctly flagged function is also explained correctly: the right vulnerability class in the right place. Let T be the set of flagged vulnerable functions that have both C_i and G_i. Function i counts as correct only when its first CWE guess is in C_i and it has at least one correct line: $\text{Joint} = \frac{1}{|T|} \sum_{i \in T} \mathbf{1}[\hat{c}_i^{(1)} \in C_i] \cdot \mathbf{1}[\hat{G}_i \cap G_i \neq \emptyset]$. Only the first CWE guess is scored, so a harness cannot earn credit by listing several candidate CWEs. Regarding parse failures, a function without a parseable verdict is counted as incorrect and stays in the denominator for all metrics, so every PrimeVul result covers all pairs.[5]

5 DISCOVERY AND TRANSFER

Motivation & Approach. We ask whether a feedback-driven design can improve vulnerability-detection harnesses over a starter seed. We consider metrics such as pairwise accuracy and localization accuracy, as a strong harness should both tell vulnerable code apart from its fix and be able to locate the flaw. However, as mentioned, a harness must balance multiple objectives at once, and a gain on one metric may result in degradation on another, affecting how the best harness is selected. We therefore explore whether our approach improves pairwise accuracy and how these improvements affect other metrics, such as localization. Gains attained on the data used for optimization could still reflect overfitting: the harness search could learn task-specific shortcuts that work on the *validation set* but disappear on the held-out tasks, or the harness can overfit to the model that wrote it, performing well on the model used for optimization but not transferring to other models (Wang et al., 2026; Wu et al., 2026). This motivates **RQ1: *Does feedback-driven search improve vulnerability-detection harnesses, and do the gains transfer to unseen data and models?*** We conduct four optimization runs using Codex CLI and Claude Code with Inkling-Small, DeepSeek V4 Flash, Opus 4.6, and GLM-5.3. Each run begins with the same SCoT seed harness, which we optimize on validation set D using Algorithm 1. At each iteration, we measure pairwise accuracy, precision, recall, CWE identification, and localization (Fig. 2). To determine the Pareto frontier, we use pairwise accuracy and joint hit rate on the validation set. Then, without changing the harnesses, we evaluate each of the harnesses on PrimeVul: first with the model used for optimization, then with a model not used during optimization. These transfer models are Nemotron 3.5 Lightning, GLM-4.7 Flash, Opus 5, and GPT-OSS-120B (Fig. 3).[8] For each evaluation, we report the best harness from the validation

[5] Errors from a parse failure are rare, occurring at 0.36% throughout and at most 0.63% on any individual run. [7] We exclude accuracy, F1, and CWE accuracy as accuracy closely follows precision, F1 incorporates findings from precision and recall, while CWE accuracy overlaps with Joint. Supp. Table S5.1 reports all metrics.
[8] Supp. S6 shows each column at full width and gives a worked example of selecting the Pareto frontier.

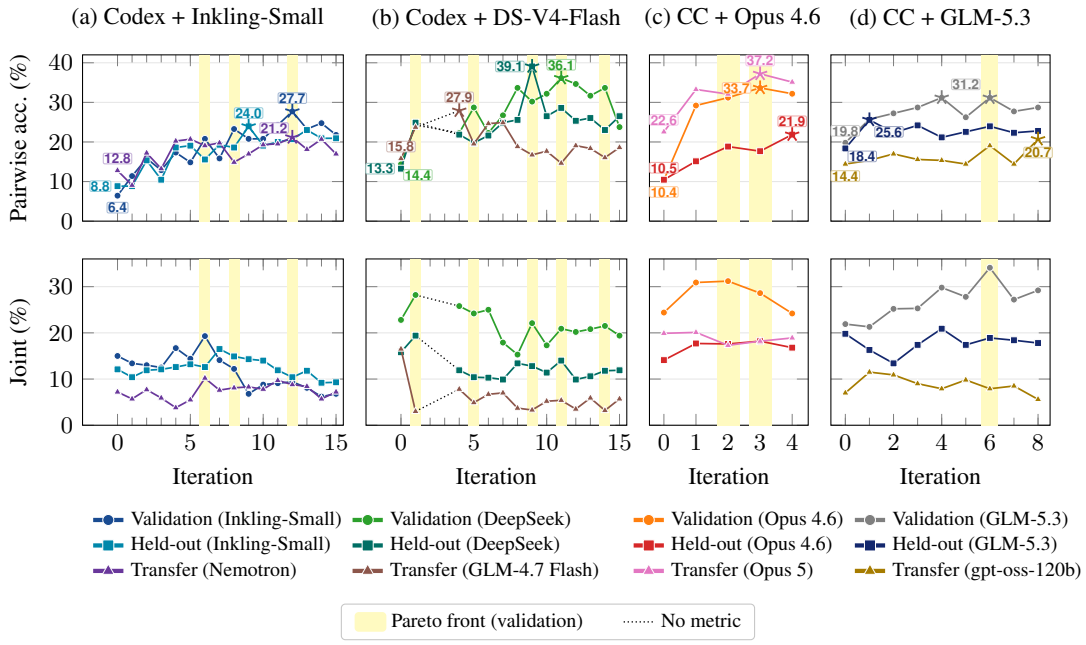


Figure 3: **Pairwise accuracy and joint hit rate across discovery iterations.** Pairwise accuracy (top) and joint hit rate (bottom) across optimizer iterations for four optimization sessions. CC = Claude Code; DS = DeepSeek. Validation uses TitanVul+CWEval; held-out and cross-model transfer use PrimeVul. ☆ marks each series' best pairwise accuracy, marks the validation Pareto front, and labels show seed and best values. Harnesses are selected using validation only; PrimeVul is never used for selection. No metric marks iterations where the optimizer failed or returned an invalid harness.

frontier. Supp. S5 reports every metric for the seed and best harness, shows how pairwise accuracy correlates with the other metrics, and shows how many harnesses beat the seed on each metric.

Results. Feedback-driven optimization improves validation pairwise accuracy in all four runs, with best-harness gains over the seed of +21.3 (Inkling-Small), +21.8 (DeepSeek V4 Flash), +23.3 (Opus 4.6) and +11.4 points (GLM-5.3) (Fig. 2), and all 40 discovered harnesses beat the seed. The gains largely carry over: a frontier harness beats the seed in all 8 held-out and cross-model evaluations (Fig. 3). On unseen PrimeVul data with the same model, the gains are +11.9, +25.8, +8.4 and +5.6 points in the same order; on models that took no part in optimization, they are +8.4 on Nemotron 3.5 Lightning, +7.9 on GLM-4.7 Flash, +14.7 on Opus 5 and +4.7 on GPT-OSS-120B. Pairwise accuracy generally increases with precision while recall decreases. Its relationship with joint hit rate differs across models, indicating a negative association for Inkling-Small and DeepSeek V4 Flash and a positive association for GLM-5.3 (Supp. Table S5.2). Only 10 of the 40 harnesses improve localization (Supp. Table S5.3). Consequently, these implications motivate the application of a Pareto frontier, rather than one specific harness chosen strictly on one of the criteria provided.

RQ1 Findings. MetaVul improves pairwise accuracy by 11.4 to 23.3 points (1.6× to 4.3×) over the seed, and the gains hold on unseen data (5.6 to 25.8 points, 1.3× to 2.9×) and unseen models (4.7 to 14.7 points, 1.3× to 1.7×). It is a critical challenge to concurrently improve discrimination, localization, and CWE identification, as singular improvements to one do not necessarily directly translate to each of the others.

6 Comparison to Other Methods

Motivation & Approach. Related work evaluates four LLM-based and agentic vulnerability detectors on PrimeVul using GPT-4o, providing a common backbone for comparison (Sec. 2). This motivates **RQ2:** ***How does MetaVul compare with other LLM-based or agentic methods?*** Because GPT-4o was costly to evaluate through OpenRouter, our budget did not permit transferring every discovered harness. We randomly selected the DeepSeek V4 Flash discovery run and evaluated the six harnesses

Table 2: **GPT-4o-based methods on PrimeVul.** P-C is the number of pairs for which both the vulnerable and patched functions are classified correctly; Error is the number of pairs in which at least one of the two functions is misclassified; and FPR is the false-positive rate. ↑ means higher is better, ↓ means lower is better, and bold marks the best result in each primary metric.

Method	P-C ↑	Error ↓	Precision ↑	Recall ↑	FPR ↓	Accuracy ↑	F1 score ↑
Ding et al.'s CoT (Ding et al., 2024)	40	395	0.5355	0.1908	0.1655	0.5126	0.2814
GPTLens (Hu et al., 2023)	44	391	0.5144	0.6552	0.6184	0.5184	0.5763
VulTrial (Widyasari et al., 2026)	81	354	0.5317	0.5977	0.5264	0.5356	0.5628
ConfColl (Tsai et al., 2025)	22	413	0.5078	0.8253	0.8000	0.5126	**0.6287**
Our approach (seed)	74	361	0.5185	0.5793	0.5379	0.5207	0.5472
Our approach (iter 12 harness)	**103**	**332**	0.5636	0.4276	0.3310	**0.5483**	0.4863
Our approach on 430 pairs, validation overlap removed[†]							
Our approach (seed)	71	359	0.5157	0.5744	0.5395	0.5174	0.5435
Our approach (iter 12 harness)	100	330	0.5631	0.4256	0.3302	0.5477	0.4848

Notes. Bold marks the best result among methods evaluated on all 435 pairs. [†]Excludes 5 pairs overlapping validation.

with the highest validation pairwise accuracy to test whether its strongest candidates transferred to GPT-4o.[9] We also evaluated iteration 7; iteration 13 did not complete. Supp. S7 reports all completed transfers, while Table 2 compares the seed with iteration 12, the transferred harness with the highest accuracy.

Results. MetaVul correctly discriminates 103 of 435 pairs, compared with 74 for the seed and 81 for the next-best method, whose results are reported from prior work (Table 2). Notably, MetaVul uses a single detector agent, whereas GPTLens, VulTrial and ConfColl coordinate multiple agents. After removing 5 pairs overlapping validation, MetaVul still correctly discriminates 100 of 430 pairs. It also achieves the highest accuracy and the lowest false-positive rate among methods with non-trivial recall. The GPT-4o evaluation was limited to a subset of discovered harnesses due to budget constraints. This result should not be considered an upper bound. The evaluation of additional harnesses may in fact yield stronger performance results.

RQ2 Findings. Using the same GPT-4o model, MetaVul achieves stronger pairwise accuracy than other existing vulnerability-detection methods. This includes multi-agent approaches, even after removing any validation overlap. These results are conclusive that the learned harness directly improves detection using a single detector agent, without ever changing the underlying model used.

7 Optimization Integrity

Motivation & Approach. Because the optimizer receives labeled validation feedback, its gains could come from shortcuts rather than better harness design. We therefore ask **RQ3:** ***Do MetaVul's gains rely on data leakage, embedded answers, or trivial verdict bias?*** We test two possible shortcut pathways. First, we verify that the optimizer cannot access PrimeVul during discovery. Second, we use GPT-OSS-120B as a judge to examine all 44 harnesses using only their text. The judge assigns numerical ratings from 0 to 4 (where 0 means missing/unusable and 4 means precise/complete) for respecting relevant constraints: (1) for preserving SCoT, (2) for avoiding prohibited content such as examples, labels, dataset hints, or verdict-bias instructions, and (3) for preserving the output format. Table 3 reports these checks for the Pareto-frontier harnesses, with full judge results in Supp. S2.

Results. We find no evidence that the Pareto-frontier harnesses exploit the tested shortcuts. All 11 receive $4/4$ for preserving SCoT and avoiding prohibited content. Two receive lower output-format ratings because of conflicting instructions, a quality issue rather than a shortcut. Every frontier harness reduces both false positives and the total number of flagged functions, with false positives falling by 33% to 60%.

[9] At the time of evaluation, GPT-4o cost $10 per million output tokens on OpenRouter.

Table 3: **Shortcut checks for Pareto-frontier harnesses.** The ratings indicate how many of the frontier harnesses receive a $4/4$ score for preserving SCoT, avoiding prohibited content, and preserving the output format. The final columns show how much the frontier harnesses cut FP (false positives) and total flagged "vulnerable" functions, compared to the run's seed. Supp. S2 shows more detailed results.

Agent + Model	Frontier iterations	SCoT	No prohibited	Output	FP reduction	Flagged reduction
Codex + Inkling-Small	6, 8, 12	3/3	3/3	3/3	33-44%	31-36%
Codex + DeepSeek V4 Flash	1, 5, 9, 11, 14	5/5	5/5	4/5	44-60%	34-49%
Claude Code + Opus 4.6	2, 3	2/2	2/2	2/2	50%	35-37%
Claude Code + GLM-5.3	6	1/1	1/1	0/1	49%	35%
Total	**11 harnesses**	**11/11**	**11/11**	**9/11**	**33-60%**	**31-49%**

RQ3 Findings. We find no evidence that MetaVul's Pareto-frontier harnesses rely on data leakage, embedded answers, or increased flagging. All reduce false positives while preserving the required reasoning scaffold.

8 Limitations

We identify some limitations and threats in this work. **Same model for both roles.** During harness discovery, the detection agent and the optimizer agent use the same model. Our cross-model transfer experiments help to mitigate this risk. We tested whether the discovered harnesses still improved detection rates on models that took *no* part in optimization, and showed that the gains carry over, hence the gains cannot be attributed to one specific model. **Seed harness.** Although we selected a strong seed based on a pilot comparison of different reasoning strategies, ultimately, the space of possible seeds is large, so it was infeasible to test whether other seeds lead to different or better harnesses. **Contribution of workspace components.** We do not establish which parts of the optimizer's workspace (earlier harnesses, aggregate metrics, the score history, or example-by-example predictions with labels) account for the improvement. A preliminary ablation that removed these components level by level was inconclusive (Supp. S11). **Language and granularity.** All experiments operate on C/C++ functions, so results may not carry over to memory-safe or dynamically typed languages. Detection is also performed at the function level, without any surrounding context or access to the rest of the repository. **Cost and access to infrastructure.** Access to API infrastructure was a severe limitation, although the results so far are very promising. Because both agents are launched as CLI processes (`codex exec` and `claude -p`), we mostly relied on subscriptions rather than per-token API billing to obtain these results. Subscriptions are limited by 5-hour and weekly usage windows, and a single optimization run, which evaluates every candidate harness on 404 functions, exhausts them very quickly. OpenRouter options for open-weight models were cheap (for example, $0.03 per million input tokens and $0.17 per million output tokens for GPT-OSS-120B; Supp. S12), but GPT-4o was not, at $2.50 per million input tokens and $10.00 per million output tokens (OpenRouter, 2026d). Due to these previously listed constraints, we were limited by the number of feasible iterations, repeated runs, and model pairings. **Data contamination.** We cannot fully exclude that the benchmarks appeared in the models' training data. Some of the evaluated models have training cutoffs that postdate the release of the benchmarks available. However, the three following observations directly suggest that this is unlikely and mitigate this concern. GPT-4o has a training cutoff (October 2023) that precedes the release of PrimeVul, TitanVul and CWEval, and harnesses learned with DeepSeek V4 Flash still improve on it (79-101 vs. 71 correct pairs; Supp. S7). The seed harness also scores near the floor on pairwise accuracy (10.5%), which memorized fixes would not produce, and grouping all 430 PrimeVul pairs by CVE year (2008-2022) shows no monotonic trend.[10]

9 Future Work

There are several directions that could be targeted by future work. First, it could be worthwhile to swap or pair different optimizer and detector agents, for example a smaller detector with a stronger

[10] We maintain that, if a model memorized fixes during training, older bugs, which have been public for longer, should be easier. Each PrimeVul pair is a real bug reported in a particular year. So, to test this, we group pairs by year (2008-2022) and measure performance in each group. We found no consistent trend across runs. Per-year results are in our CVE-year report.

optimizer, or Claude Code with Codex, to measure how much each role contributes to the discovered harnesses. The trade-off between discrimination, localization, and CWE identification should also be studied directly, since techniques that improve these metrics together remain underexplored. Starting discovery from a different seed harness could also show how the seed limits or biases evolution downstream. We can also think of many enhancements to the harness itself, such as a spec-driven approach to harness design, where a declarative specification can outline the input and output format, constrain each candidate harness, and check it before evaluation. The detector could also be given other context, like pre-extracted data-flow and program dependence graphs, or call graphs and caller/callee information (using the whole project), which can be pre-computed for the data and included in the workspace. Finally, direct integration into agentic exploit generation could be explored, which has implications for improving cost and efficiency.

10 Conclusion

MetaVul automatically discovers vulnerability-detection harnesses that outperform a hand-designed seed. Across four runs, validation pairwise accuracy improves by 11.4-23.3 points ($1.6\times$-$4.3\times$). These gains persist across all eight held-out and cross-model evaluations (up to 25.8 points on unseen PrimeVul data and 14.7 on an unseen model). On GPT-4o, a transferred harness correctly discriminates 103 of 435 pairs, outperforming existing methods and achieving the highest accuracy. We find no evidence of shortcuts: every Pareto-frontier harness preserves SCoT, avoids prohibited content, and reduces false positives by 33%-60%. MetaVul thus addresses two challenges. Firstly, it *reduces false alarms*, cutting false positives while improving discrimination between security-relevant changes and benign code variations. Secondly, it *improves generalization*, showing that the gains hold on unseen data and unseen models rather than overfitting.

These results have further implications for real-world vulnerability detection, including, but not limited to, analyzing new codebases, automated exploit generation, and triage. First, the transferability results show that this technique can transfer to unseen codebases. Second, it can make automated exploit generation more efficient and scalable: it can filter out benign code from promising vulnerabilities before exploit generation and execution, which are computationally expensive (see Future Work, Sec. 9). Even after exploit generation, people must decide how serious an exploit is, what its potential downstream effects are, and how quickly a fix should be applied. Thus, there are major implications for security auditing and triage, as this technique can reduce the need for developer intervention.

Reproducibility Statement

Our code has been publicly available at `https://github.com/ARiSE-Lab/MetaVul` (ARiSE Lab, 2026) since September 30, 2026. This work, from conception to implementation, took place between February and September 2026.

Acknowledgements

Part of this work was supported by a Tinker Grant from Thinking Machines Lab (TML). We thank TML for their support. We also thank Ciaran Ramasastry Walsh for his discussions and early explorations in the Codex environment.

AI Use Statement

We used AI to improve transitions and wording in our writing to make it clearer for our audience in cases where page limit constraints are a concern. We used our hand-written drafts prior to improving transitions or wording. We did not use AI to conceive any research ideas. All ideas came from reading the literature and related work, discussions among the authors, and insights from past projects, and we first wrote our own notes and human-authored drafts. We also used Claude Code and Codex for code maintenance and debugging. All AI-assisted text and code were reviewed by the authors.

REFERENCES

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2026. URL https://arxiv.org/abs/2507.19457.

ARiSE Lab. MetaVul: Code repository. https://github.com/ARiSE-Lab/MetaVul, sep 2026. GitHub repository, made public September 30, 2026.

Nicholas Carlini, Keane Lucas, Evyatar Ben Asher, Newton Cheng, Hasnain Lakhani, David Forsythe, and Kyla Guru. Evaluating and mitigating the growing risk of LLM-discovered 0-days. Anthropic, February 2026. URL https://www.anthropic.com/research/zero-days.

Wachiraphan Charoenwet, Patanamon Thongtanunam, Van-Thuan Pham, and Christoph Treude. An empirical study of static analysis tools for secure code review, 2024. URL https://arxiv.org/abs/2407.12241.

Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models: How far are we?, 2024. URL https://arxiv.org/abs/2403.18624.

Esakkivel Esakkiraja, Denis Akhiyarov, Vikas Yadav, Sai Rajeswar, Patrice Bechard, Sridhar Nemala, and Sagar Davasam. Starharness: Evolving harnesses with stratified search for enterprise environments, 2026. URL https://arxiv.org/abs/2608.24804.

Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model, 2024. URL https://arxiv.org/abs/2403.07691.

Sihao Hu, Tiansheng Huang, Fatih İlhan, Selim Furkan Tekin, and Ling Liu. Large language model-powered smart contract vulnerability detection: New perspectives. In *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, pp. 297–306, 2023. doi: 10.1109/TPS-ISA58951.2023.00044.

Lisheng Huang, Chen Yang, Hao Zhou, Huatong Song, Zongchao Chen, Ran Le, Yang Song, Wayne Xin Zhao, and Tao Zhang. Evo-bench: Can language models improve agent harness?, 2026a. URL https://arxiv.org/abs/2608.09096.

Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. Memoharness: Agent harnesses that learn from experience, 2026b. URL https://arxiv.org/abs/2607.14159.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL https://arxiv.org/abs/2603.28052.

Fengjie Li, Jiajun Jiang, Dongchi Chen, and Yingfei Xiong. Llm-based vulnerability detection at project scale: An empirical study, 2026a. URL https://arxiv.org/abs/2601.19239.

Jia Li, Ge Li, Yongmin Li, and Zhi Jin. Structured chain-of-thought prompting for code generation, 2023. URL https://arxiv.org/abs/2305.06599.

Yikun Li, Ngoc Tan Bui, Ting Zhang, Chengran Yang, Xin Zhou, Martin Weyssow, Jinfeng Jiang, Junkai Chen, Huihui Huang, Huu Hung Nguyen, et al. Out of distribution, out of luck: How well can llms trained on vulnerability datasets detect top 25 cwe weaknesses? In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, pp. 2096–2108, 2026b.

Houjun Liu, Lisa Einstein, John Yang, Joachim Baumann, Duncan Eddy, Christopher D. Manning, Mykel Kochenderfer, and Diyi Yang. Secureforge: Finding and preventing vulnerabilities in llm-generated code via prompt optimization, 2026. URL https://arxiv.org/abs/2605.08382.

Ryan Lopopolo. Harness engineering: Leveraging Codex in an agent-first world. OpenAI, https://openai.com/index/harness-engineering/, February 2026.

Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. Tthe: Test-time harness evolution, 2026. URL https://arxiv.org/abs/2607.08124.

OpenAI. GPT-4o system card, 2024. URL https://arxiv.org/abs/2410.21276.

OpenRouter. DeepSeek: DeepSeek V4 Flash model pricing. https://openrouter.ai/deepseek/deepseek-v4-flash-0731, 2026a.

OpenRouter. Z.ai: GLM-5.3 model pricing. https://openrouter.ai/z-ai/glm-5.3, 2026b.

OpenRouter. Z.ai: GLM 4.7 Flash model pricing. https://openrouter.ai/z-ai/glm-4.7-flash, 2026c.

OpenRouter. OpenAI: GPT-4o model pricing. https://openrouter.ai/openai/gpt-4o, 2026d.

OpenRouter. OpenAI: gpt-oss-120b model pricing. https://openrouter.ai/openai/gpt-oss-120b, 2026e.

OpenRouter. NVIDIA: Nemotron 3.5 Lightning model pricing. https://openrouter.ai/nvidia/nemotron-3.5-lightning, 2026f.

Ruwei Pan and Hongyu Zhang. MoT: Modularization-of-thought prompting for effective code generation, 2025. URL https://arxiv.org/abs/2503.12483.

Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. Cweval: Outcome-driven evaluation on functionality and security of llm code generation. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, pp. 33–40. IEEE, 2025.

Prithvi Rajasekaran. Harness design for long-running application development. Anthropic, https://www.anthropic.com/engineering/harness-design-long-running-apps, March 2026.

Amirali Sajadi, Saikat Dutta, and Preetha Chatterjee. An empirical analysis of CodeQL false positives and query refinements for Java vulnerabilities, 2026. URL https://arxiv.org/abs/2609.04535.

Wenxin Tang, Xiang Zhang, Junliang Liu, Jingyu Xiao, Xi Xiao, Jinlong Yang, Yuehe Ma, Zhenyu Liu, Zhengheng Li, Zicheng Wang, Wang Luo, Qing Li, Lei Wang, and Peng Xiangli. VulTriage: Triple-path context augmentation for LLM-based vulnerability detection, 2026. URL https://arxiv.org/abs/2605.09461.

Shahroz Tariq, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. Alert fatigue in security operations centres: Research challenges and opportunities. *ACM Comput. Surv.*, 57(9), April 2025. ISSN 0360-0300. doi: 10.1145/3723158. URL https://doi.org/10.1145/3723158.

Thinking Machines Lab. Models & pricing. Tinker documentation, https://tinker-docs.thinkingmachines.ai/tinker/models/, 2026.

Chung-Nan Tsai, Xin Wang, Cheng-Hsiung Lee, and Ching-Sheng Lin. A sequential multi-stage approach for code vulnerability detection via confidence- and collaboration-based decision making. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 21151–21157, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1071. URL https://aclanthology.org/2025.emnlp-main.1071/.

Varun Ursekar, Apaar Shanker, Yash Maurya, Shehab Yasser, Vijay S. Kalmath, Veronica Chatrath, and Yuan Xue. Harnessopt-bench: Evaluating llms at harness optimization, 2026. URL https://arxiv.org/abs/2608.06301.

John Walker. Benchmarking Opus 4.6 for vuln detection: Flashes of brilliance but lots of noise. ZeroPath, https://zeropath.com/blog/benchmarking-opus-4-6-vuln-detection, April 2026.

Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, 2026. URL https://arxiv.org/abs/2607.12227.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Ratnadira Widyasari, Martin Weyssow, Ivana Clairine Irsan, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo. Let the trial begin: A mock-court approach to vulnerability detection using llm-based agents. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, ICSE '26, pp. 1030–1042, New York, NY, USA, 2026. Association for Computing Machinery. ISBN 9798400720253. doi: 10.1145/3744916.3773256. URL https://doi.org/10.1145/3744916.3773256.

Wiz Research. PHP2Shell: AI uncovers critical RCE vulnerabilities affecting 70% of the internet. https://www.wiz.io/blog/php2shell-php-rce, October 2026. Accessed October 26, 2026.

Yuhao Wu, Jingyuan Zhang, Jiajun Shi, Xinping Lei, Qingshui Gu, Yuxuan Zhang, Zexuan Wang, Chen He, Chen Huang, Maojia Song, Zhiyuan Zeng, Shaowen Wang, Jinkai Liu, Yunfeng Shi, Jiaheng Liu, Shen Yan, Wenhao Huang, Ge Zhang, and Wenxuan Zhang. Harnessdev: Can llms create and evolve their own agent harness?, 2026. URL https://arxiv.org/abs/2609.01437.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pp. 11809–11822, 2023. doi: 10.52202/075280-0517. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract-Conference.html.

Alperen Yildiz, Sin G. Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil M. Divakaran. Benchmarking llms and llm-based agents in practical vulnerability detection for code repositories, 2025. URL https://arxiv.org/abs/2503.03586.

Yifan Zhang, Yutong Dai, Juntao Tan, Luyu Yang, Rishi Mullur, Thai Hoang, Zhiyuan Hu, James Zhu, Phil Mui, Silvio Savarese, Ran Xu, and Zeyuan Chen. Darwinx: Evolving agent harnesses through natural selection, 2026. URL https://arxiv.org/abs/2608.07545.

Jingsheng Zheng, Xinyuan Fang, Jintian Zhang, Zhengke Gui, Huajun Chen, and Ningyu Zhang. Onedayagent: Towards a long-horizon harness for autonomous agents, 2026. URL https://arxiv.org/abs/2608.05013.